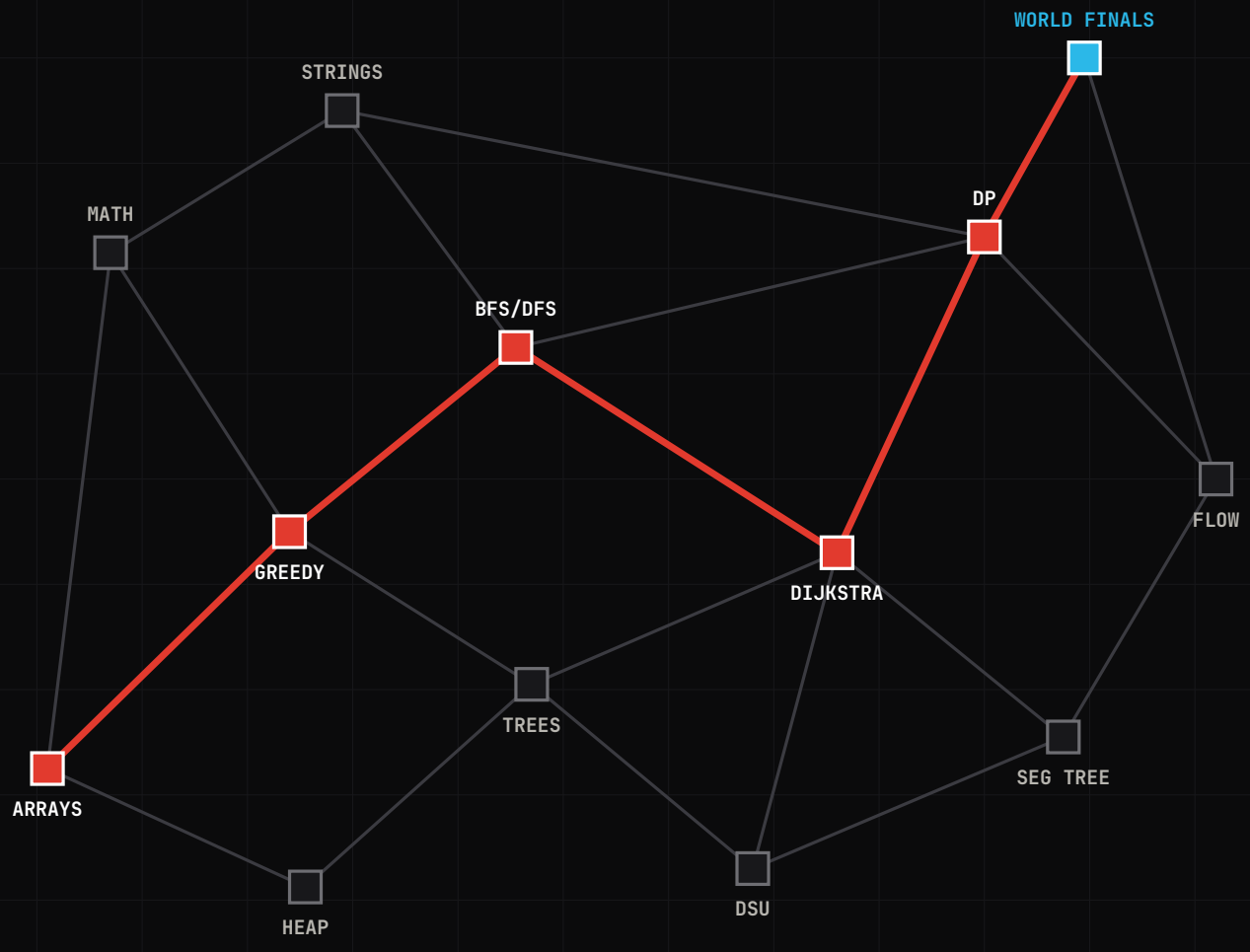
MINDFLARE / IT DEPARTMENT SOCIETY

COMPETITIVE PROGRAMMING SYLLABUS



ICPC: from first contest to World Finals.

16 modules / 84 sections / C++, graphs, DP, math, strings, geometry, team play

MINDFLARE

where algorithm sparks intuition

2026 EDITION

About This Syllabus

A complete, module-by-module syllabus for competitive programming, built for the ICPC route from your first contest all the way to World Finals. It starts with C++ and contest mechanics, moves through the core data structures, graphs and dynamic programming, and finishes with advanced structures, mathematics, strings, geometry and team strategy.

How It Is Organized

- 16 modules, each split into numbered sections (for example 3.1, 8.4, 11.7).
- Most modules end with Problem-Solving Patterns: a recognition cue ("Can I reach it?") paired with the technique to reach for (DFS / BFS).
- Cross-references such as Module 10.5 or Module 14 show where topics connect across modules.
- Notes like conceptual, advanced or optional mark depth: know the idea first, implement it later.
- The suggested progression at the end of this document shows the order to study the modules in.

> **HOW TO USE IT**
> Tick a topic off only when you can solve problems on it without hints and explain the idea aloud. Upsolve every contest and tag each problem with its module number.

© 2026 Mindflare · IT Department Society

Module Map

No.	Module	Sections	Topics
01	Language & Environment	2	13
02	Complexity & Problem-Solving Foundations	2	7
03	Arrays & Prefix Techniques	4	27
04	Recursion, Backtracking & Bit Manipulation	4	18
05	Greedy Algorithms	3	11
06	Core Data Structures	7	78
07	Trees	4	35
08	BFS / DFS	12	90
09	Weighted Graphs & Shortest Paths	5	21
10	Advanced Graph Theory	6	33
11	Dynamic Programming	11	54
12	Advanced Data Structures	6	26
13	Mathematics for Competitive Programming	7	33
14	String Algorithms	4	16
15	Computational Geometry	4	14
16	ICPC Team & Contest Skills	3	12
	Total	84	488

© 2026 Mindflare · IT Department Society

Table of Contents

About This Syllabus . . . 2

01 Language & Environment . . . 5

02 Complexity & Problem-Solving Foundations . . . 5

03 Arrays & Prefix Techniques . . . 5

04 Recursion, Backtracking & Bit Manipulation . . . 6

05 Greedy Algorithms . . . 7

06 Core Data Structures . . . 8

07 Trees . . . 10

08 BFS / DFS . . . 11

09 Weighted Graphs & Shortest Paths . . . 13

10 Advanced Graph Theory . . . 14

11 Dynamic Programming . . . 15

12 Advanced Data Structures . . . 17

13 Mathematics for Competitive Programming . . . 18

14 String Algorithms . . . 20

15 Computational Geometry . . . 20

16 ICPC Team & Contest Skills . . . 21

Suggested Progression . . . 22

© 2026 Mindflare · IT Department Society

01 Language & Environment

2 SECTIONS / 13 TOPICS

1.1 C++ for Competitive Programming

- Fast I/O (sync_with_stdio(false), cin.tie(nullptr))
- Data types, overflow, long long vs int
- STL: vector, pair, tuple, array
- STL: set, map, unordered_set, unordered_map
- STL: stack, queue, deque, priority_queue
- Iterators, ranges, auto, structured bindings
- Custom comparators, lambdas
- Templates & generic functions

1.2 Contest Mechanics

- Reading problem statements (constraints → algorithm hints)
- Time limits → complexity budget estimation
- Stress testing / brute-force comparison scripts
- Debugging under pressure
- Editorial reading & upsolving discipline

02 Complexity & Problem-Solving Foundations

2 SECTIONS / 7 TOPICS

2.1 Complexity Analysis

- Big-O, Big-Θ, Big-Ω
- Time vs space tradeoffs
- Amortized analysis
- Estimating feasible complexity from constraints (1e5 → O(n log n), 1e9 → O(log n)/O(1))

2.2 Problem-Solving Patterns

- Brute force → optimize pipeline
- Identifying problem "shape" from constraints
- Working backwards from output to algorithm

03 Arrays & Prefix Techniques

4 SECTIONS / 27 TOPICS

© 2026 Mindflare · IT Department Society

3.1 Array Fundamentals

- 1D / 2D Arrays
- Array Rotation (Left/Right, In-Place Reversal Trick)
- Kadane's Algorithm (Max Subarray Sum)
- Kadane's Variants (Max Circular Subarray, Max Product Subarray)
- Dutch National Flag (3-Way Partitioning / Sort 0s-1s-2s)
- Moore's Voting Algorithm (Majority Element)
- Prefix Sums
- Difference Arrays
- 2D Prefix Sums
- Suffix Sums
- Matrix Traversal Patterns (Spiral, Diagonal, Zigzag)
- Matrix Rotation In-Place (90°)
- Set Matrix Zeroes / In-Place Matrix Marking

3.2 Two Pointers & Sliding Window

- Fixed-Size Window
- Variable-Size Window
- Two Pointers (opposite ends)
- Two Pointers (same direction)
- Merging Two Sorted Arrays

3.3 Sorting & Searching

- Sorting Algorithms (Comparison-Based)
- Custom Comparators
- Binary Search on Array
- Binary Search on Answer
- Ternary Search (unimodal functions)

3.4 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Subarray Sum Equals K?	**Prefix Sum + Hashing**
Range Update, Point Query?	**Difference Array**
Monotonic Property in Answer?	**Binary Search on Answer**
Contiguous Window with Condition?	**Sliding Window**

04 Recursion, Backtracking & Bit Manipulation

4 SECTIONS / 18 TOPICS

4.1 Recursion

- Base Cases & Recursive Cases
- Recursion Tree / Call Stack Visualization
- Recursion → Iteration Conversion
- Divide and Conquer

© 2026 Mindflare · IT Department Society

4.2 Backtracking

- Generating Subsets
- Generating Permutations
- Combinations
- N-Queens Style Pruning
- Constraint Satisfaction

4.3 Bit Manipulation

- Binary Representation
- AND / OR / XOR / NOT / Shifts
- Popcount, Lowest Set Bit
- Bitmasks for Subsets
- XOR Tricks (finding unique element, pairing)
- Bitwise DP Preview

4.4 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
All Subsets/Permutations?	**Backtracking**
Toggle / Parity Tricks?	**XOR**
Small n (≤20)?	**Bitmask Enumeration**

05 Greedy Algorithms

3 SECTIONS / 11 TOPICS

5.1 Greedy Foundations

- Exchange Argument (proving greedy correctness)
- Activity Selection
- Interval Scheduling
- Interval Merging
- Interval Point Covering

5.2 Greedy on Structures

- Huffman Encoding
- Fractional Knapsack
- Greedy with Sorting
- Greedy with Priority Queue

© 2026 Mindflare · IT Department Society

5.3 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Maximize/Minimize Count of Non-Overlapping?	**Sort + Greedy**
Local Optimal Choice Provably Global?	**Exchange Argument**

06 Core Data Structures

7 SECTIONS / 78 TOPICS

6.1 Linked Lists

- Singly Linked List: Insert, Delete, Traverse
- Doubly Linked List: Insert, Delete, Traverse
- Circular Linked List
- Reversing a Linked List (Iterative)
- Reversing a Linked List (Recursive)
- Reversing in Groups of K
- Finding the Middle Node (Slow/Fast Pointer)
- Cycle Detection (Floyd's Tortoise and Hare)
- Finding Cycle Start Point
- Merging Two Sorted Linked Lists
- Merging K Sorted Linked Lists
- Removing N-th Node from End
- Detecting & Removing Duplicates
- Intersection Point of Two Linked Lists
- Palindrome Linked List Check
- Sorting a Linked List (Merge Sort on LL)
- Flattening a Multilevel Linked List
- Copying a List with Random Pointers
- Design: LRU Cache (Doubly LL + Hash Map)
- Design: LFU Cache

6.2 Stacks

- Stack via Array / Linked List
- Balanced Parentheses / Bracket Matching
- Min Stack (O(1) get-min)
- Stack with getMin/getMax in O(1)
- Infix to Postfix / Prefix Conversion
- Evaluating Postfix / Prefix Expressions
- Next Greater Element (Monotonic Stack)
- Next Smaller Element
- Previous Greater/Smaller Element
- Largest Rectangle in Histogram
- Maximal Rectangle in Binary Matrix
- Stock Span Problem
- Trapping Rain Water (Stack Approach)
- Sorting a Stack (using recursion/auxiliary stack)
- Design: Stack using Queues
- Design: Queue using Stacks
- Celebrity Problem

6.3 Queues & Deques

- Queue via Array (Circular Queue) / Linked List
- Deque Operations (push/pop both ends)
- Monotonic Deque
- Sliding Window Maximum/Minimum (Monotonic Deque)
- Circular Buffers
- Design: Moving Average from Data Stream
- Priority Queue vs Deque vs Stack — When to Use Which

© 2026 Mindflare · IT Department Society

6.4 Hashing

- Hash Maps / Sets Usage Patterns
- Collision Handling (Chaining, Open Addressing) — conceptual
- Custom Hash Functions (anti-hash test defense on Codeforces)
- Coordinate Compression
- Frequency Counting Patterns
- Two-Sum Style Complement Lookups
- Grouping by Hash Key (Anagrams, etc.)
- Rolling Hash (bridge to String Algorithms, Module 14)

6.5 Heaps

- Binary Heap Implementation (Array-Based)
- Heapify (Build Heap in O(n))
- Priority Queue Basics
- Min-Heap / Max-Heap
- K-th Largest/Smallest Element
- Top-K Frequent Elements
- Merging K Sorted Lists/Arrays
- Median from a Data Stream (Two-Heap Technique)
- Heap + Greedy Combination
- Indexed/Updatable Priority Queue (conceptual)

6.6 Disjoint Set Union (DSU)

- Union by Rank/Size
- Path Compression
- DSU with Extra Data (component size, sum, parity)
- Offline Connectivity Queries
- DSU on Grid / Kruskal Integration
- Weighted/Union-Find with Relations (parity DSU)
- Rollback DSU (undo operations, offline dynamic connectivity)

6.7 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Reverse/Reorder a List In-Place?	**Linked List Pointer Manipulation**
Detect a Cycle?	**Fast/Slow Pointers**
Next Greater/Smaller Element?	**Monotonic Stack**
Sliding Window Max/Min?	**Monotonic Deque**
Valid Expression / Matching Brackets?	**Stack**
K-th Smallest/Largest, or Running Top-K?	**Heap**
Design an O(1) Cache?	**Linked List + Hash Map (LRU)**
Dynamic Connectivity / Grouping?	**DSU**
Complement/Pair Lookup Fast?	**Hash Map**

© 2026 Mindflare · IT Department Society

07 Trees

4 SECTIONS / 35 TOPICS

7.1 Tree Fundamentals

- Tree Terminology (root, leaf, depth, height)
- Binary Trees
- Binary Search Trees (BST)
- Tree Traversals (Preorder, Inorder, Postorder, Level-order) — Recursive
- Tree Traversals — Iterative (using explicit stack)
- Morris Traversal (O(1) space inorder)
- Tree Representation (adjacency list for general trees)
- Constructing Tree from Traversals (Preorder+Inorder, etc.)
- Serialize / Deserialize a Binary Tree

7.2 Binary Search Tree Operations

- Insert / Delete / Search in BST
- Validate a BST
- Floor / Ceiling in BST
- K-th Smallest/Largest in BST
- Inorder Successor / Predecessor
- Convert BST to Sorted Doubly Linked List
- Balanced BST Concepts (AVL, Red-Black — conceptual only)

7.3 Tree Algorithms

- Diameter of a Tree
- Height / Depth Calculation
- Subtree Size Calculation
- Lowest Common Ancestor — Naive (Parent Pointers / Path Storage)
- Lowest Common Ancestor — Binary Lifting (bridge to Module 10.5)
- Tree DP Introduction
- Rerooting Technique
- Vertical Order / Level-Order Variants (Zigzag, Right View, Left View)
- Maximum Path Sum in a Binary Tree
- Lowest Common Ancestor in BST (optimized using BST property)
- Boundary Traversal of a Tree
- Symmetric Tree / Mirror Check
- Balanced Tree Check (height-balanced)

7.4 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Path Between Two Nodes?	**LCA**
Aggregate Over Subtree?	**DFS + Tree DP**
Answer Depends on Every Node as Root?	**Rerooting**
Reconstruct Tree from Traversal Arrays?	**Recursive Construction**
K-th Element in Sorted Order?	**Inorder Traversal on BST**
O(1) Space Traversal?	**Morris Traversal**

© 2026 Mindflare · IT Department Society

08 BFS / DFS

12 SECTIONS / 90 TOPICS

8.1 Graph Representation & Traversal

- Graph Terminology: Vertices, Edges, Degree
- Directed vs Undirected Graphs
- Weighted vs Unweighted Graphs
- Adjacency Matrix
- Adjacency List
- Edge List
- Recursive DFS
- Iterative DFS
- BFS with Queue
- Time & Space Complexity

8.2 Basic Graph Applications

- Connected Components
- Counting Components
- Reachability
- Finding a Path
- Number of Paths
- Graph Traversal from Multiple Sources
- Handling Disconnected Graphs

8.3 Grid BFS / DFS

- Grid Representation
- 4-Direction and 8-Direction Movement
- Flood Fill
- Connected Components in a Grid
- Counting Islands / Regions
- Boundary Traversal
- Multi-Source BFS on Grids

8.4 BFS Shortest Path

- Shortest Path in an Unweighted Graph
- BFS Distance Array
- Parent / Path Reconstruction
- Level-Order BFS
- Multi-Source BFS
- State-Space BFS
- BFS on Implicit Graphs

8.5 DFS Applications

- Entry and Exit Times
- Parent Tracking
- Depth of Nodes
- Subtree Sizes
- Tree Traversal with DFS
- Detecting Cycles
- Path Reconstruction
- DFS on Directed Graphs
- DFS on Undirected Graphs

© 2026 Mindflare · IT Department Society

8.6 Bipartite Graphs

- Bipartite Graph Definition
- Checking Bipartiteness with BFS
- Checking Bipartiteness with DFS
- 2-Coloring
- Odd Cycle Detection
- Bipartite Components

8.7 Cycle Detection

- Cycle Detection in Undirected Graphs
- Cycle Detection with DFS
- Cycle Detection with BFS
- Cycle Detection in Directed Graphs
- Three-State DFS
- Back Edges
- Detecting Cycles in Functional Graphs

8.8 Topological Sorting

- DAG (Directed Acyclic Graph)
- DFS-Based Topological Sort
- Kahn's Algorithm
- Indegree-Based BFS
- Cycle Detection Using Topological Sort
- Lexicographically Smallest Topological Order
- Dependency / Prerequisite Problems

8.9 Advanced BFS Techniques

- 0-1 BFS
- Multi-Source BFS
- Bidirectional BFS
- BFS with Extra State
- BFS on Product Graphs
- BFS with Bitmask State
- BFS with Parity / Time / Phase States

8.10 Advanced DFS Techniques

- DFS Tree
- Subtree Processing
- Rerooting Basics
- Euler Tour / DFS Ordering
- Flattening a Tree
- Entry / Exit Time Intervals
- Offline Subtree Queries

© 2026 Mindflare · IT Department Society

8.11 Functional Graphs

- Functional Graph Representation
- Finding Cycles
- Cycle Entry Detection
- Distance to Cycle
- Binary Lifting Introduction
- Jumping Along Functional Graphs
- Cycle + Tree Decomposition

8.12 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Can I Reach It?	**DFS/BFS**
Minimum Moves?	**BFS**
Count Regions?	**DFS/BFS**
Dependencies?	**Topological Sort**
Two Groups?	**Bipartite Check**
Cycle Exists?	**DFS / Kahn's Algorithm**
Multiple Starting Points?	**Multi-Source BFS**
Minimum Cost with 0/1 Edges?	**0-1 BFS**
State Changes After Each Move?	**State-Space BFS**

09 Weighted Graphs & Shortest Paths

5 SECTIONS / 21 TOPICS

9.1 Single-Source Shortest Path

- Dijkstra's Algorithm (heap-based)
- Dijkstra with Path Reconstruction
- Bellman-Ford Algorithm
- Detecting Negative Cycles
- SPFA (Queue-Optimized Bellman-Ford)

9.2 All-Pairs Shortest Path

- Floyd-Warshall Algorithm
- Transitive Closure
- Detecting Negative Cycles with Floyd-Warshall

9.3 Minimum Spanning Tree

- Kruskal's Algorithm (with DSU)
- Prim's Algorithm
- MST Uniqueness / Second-Best MST
- Maximum Spanning Tree

© 2026 Mindflare · IT Department Society

9.4 Shortest Path Variants

- Shortest Path with State (K-stops, K-edges)
- Shortest Path in DAG (Topological Order)
- Shortest Path Counting
- Shortest Path with Modified Weights (Johnson's Algorithm)

9.5 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Non-Negative Weights, Shortest Path?	**Dijkstra**
Negative Weights Possible?	**Bellman-Ford**
All Pairs, Small n?	**Floyd-Warshall**
Connect Everything, Minimum Cost?	**MST**
Shortest Path in a DAG?	**Topo Sort + DP**

10 Advanced Graph Theory

6 SECTIONS / 33 TOPICS

10.1 Connectivity

- Articulation Points (Cut Vertices)
- Bridges (Cut Edges)
- Biconnected Components
- Strongly Connected Components — Tarjan's Algorithm
- Strongly Connected Components — Kosaraju's Algorithm
- Condensation Graph (SCC → DAG)

10.2 Euler & Hamiltonian Paths

- Euler Path / Circuit Conditions
- Hierholzer's Algorithm
- Hamiltonian Path (small n, DP-based)

10.3 Flow Networks

- Max Flow Concept (Source, Sink, Capacity)
- Ford-Fulkerson Method
- Edmonds-Karp Algorithm
- Dinic's Algorithm
- Min-Cut Max-Flow Theorem
- Min-Cost Max-Flow
- Flow with Lower Bounds

© 2026 Mindflare · IT Department Society

10.4 Matching

- Bipartite Matching (Augmenting Path)
- Hopcroft-Karp Algorithm
- Konig's Theorem
- Hungarian Algorithm (Assignment Problem)
- General Graph Matching (Blossom Algorithm — advanced/optional)

10.5 Special Techniques

- 2-SAT
- Binary Lifting for LCA
- Euler Tour + Sparse Table LCA
- Heavy-Light Decomposition
- Centroid Decomposition
- Small-to-Large Merging on Trees

10.6 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Critical Edges/Vertices?	**Bridges/Articulation Points**
Group Mutually Reachable Nodes?	**SCC**
Maximum Matching?	**Bipartite Matching / Flow**
Resource Allocation with Capacities?	**Max Flow**
Boolean Constraints (A or B)?	**2-SAT**
Repeated Path Queries on Tree?	**LCA / HLD**

11 Dynamic Programming

11 SECTIONS / 54 TOPICS

11.1 DP Foundations

- Overlapping Subproblems & Optimal Substructure
- Memoization (Top-Down)
- Tabulation (Bottom-Up)
- State Definition & Transition Design
- Reconstructing the DP Answer (path/choice tracking)

11.2 1D & 2D DP

- Fibonacci-Style DP
- Climbing Stairs Variants
- Longest Increasing Subsequence ($O(n^2)$ and $O(n \log n)$)
- Maximum Subarray (Kadane's)
- Grid Path DP

© 2026 Mindflare · IT Department Society

11.3 Knapsack Family

- 0/1 Knapsack
- Unbounded Knapsack
- Bounded Knapsack
- Subset Sum
- Partition Problems
- Coin Change (Ways / Min Coins)

11.4 String / Sequence DP

- Longest Common Subsequence (LCS)
- Edit Distance
- Longest Palindromic Subsequence
- Palindrome Partitioning DP
- Distinct Subsequences
- Wildcard/Regex Matching DP

11.5 Interval DP

- Matrix Chain Multiplication
- Palindrome Partitioning (Min Cuts)
- Burst Balloons Style DP
- Optimal BST

11.6 DP on Trees

- Subtree DP
- Diameter via Tree DP
- Rerooting DP
- DP with Merging Children States
- Independent Set on Trees

11.7 DP on Graphs / Bitmask DP

- DP on DAG (Longest Path)
- Traveling Salesman Problem (Bitmask DP)
- Assignment Problem via Bitmask
- Broken Profile DP (Tiling Problems)
- DP with Subset States

11.8 Digit DP

- Counting Numbers with Digit Constraints
- Digit DP with Tight/Loose Bound State
- Digit DP with Sum/Mod Constraints

© 2026 Mindflare · IT Department Society

11.9 Probability & Expectation DP

- Expected Value Basics
- Linearity of Expectation in DP
- Markov Chain-Style DP
- Random Walk DP

11.10 DP Optimization Techniques

- Divide and Conquer Optimization
- Convex Hull Trick / Li Chao Tree
- Knuth's Optimization
- Monotonic Queue Optimization
- Slope Trick

11.11 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Choices with Constraints, Optimize?	**Knapsack-Style DP**
Compare Two Sequences?	**LCS/Edit Distance Style DP**
Small n (≤20), Visit All?	**Bitmask DP**
Count Numbers in Range with Property?	**Digit DP**
Aggregate Over Tree?	**Tree DP**
DP Transition is O(n) but Monotonic?	**DP Optimization**

12 Advanced Data Structures

6 SECTIONS / 26 TOPICS

12.1 Segment Trees

- Point Update, Range Query
- Range Update, Point Query
- Lazy Propagation (Range Update, Range Query)
- Segment Tree Beats
- Merge Sort Tree
- Persistent Segment Tree
- Segment Tree on Implicit/Dynamic Ranges
- 2D Segment Tree

12.2 Fenwick Tree (BIT)

- Point Update, Prefix Query
- Range Update, Range Query (via 2 BITs)
- 2D Fenwick Tree
- BIT for Order Statistics (K-th Element)

© 2026 Mindflare · IT Department Society

12.3 Sqrt Decomposition

- Block Decomposition for Range Queries
- Mo's Algorithm (Offline Range Queries)
- Mo's Algorithm with Updates
- Mo's Algorithm on Trees

12.4 Sparse Structures

- Sparse Table (RMQ, static queries)
- Disjoint Sparse Table

12.5 Trie

- Basic Trie (Insert/Search)
- Trie for Prefix Queries
- Bitwise Trie (XOR Maximum/Minimum Queries)
- Trie with Counting

12.6 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Range Query + Range Update?	**Segment Tree with Lazy Propagation**
Point Update, Prefix Sum?	**Fenwick Tree**
Offline Queries, No Updates?	**Mo's Algorithm / Sparse Table**
Maximize XOR Pair?	**Bitwise Trie**

13 Mathematics for Competitive Programming

7 SECTIONS / 33 TOPICS

13.1 Number Theory Basics

- GCD / LCM (Euclidean Algorithm)
- Extended Euclidean Algorithm
- Modular Arithmetic
- Modular Inverse (Fermat's Little Theorem, Extended Euclid)
- Sieve of Eratosthenes
- Segmented Sieve
- Prime Factorization (Trial Division, Pollard's Rho)

13.2 Number Theoretic Functions

- Euler's Totient Function
- Divisor Functions (count/sum of divisors)
- Mobius Function
- Chinese Remainder Theorem

© 2026 Mindflare · IT Department Society

13.3 Combinatorics

- Permutations & Combinations
- Pascal's Triangle
- Factorials & Modular Factorials
- Inclusion-Exclusion Principle
- Catalan Numbers
- Stars and Bars
- Derangements

13.4 Algebra & Exponentiation

- Fast/Binary Exponentiation
- Matrix Exponentiation (linear recurrences)
- Gaussian Elimination (linear systems, XOR basis)
- Linear Recurrences (Berlekamp-Massey — advanced)

13.5 Game Theory

- Nim Game
- Sprague-Grundy Theorem
- Grundy Numbers for Game States
- Game DP

13.6 Polynomial & Transform Techniques

- FFT (Fast Fourier Transform) for Polynomial Multiplication
- NTT (Number Theoretic Transform)
- Convolution Applications

13.7 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Count Ways Modulo Prime?	**Combinatorics + Modular Inverse**
Large Power / Recurrence?	**Fast Exponentiation / Matrix Exponentiation**
Who Wins the Game?	**Grundy Numbers**
Multiply Large Polynomials Fast?	**FFT/NTT**

© 2026 Mindflare · IT Department Society

14 String Algorithms

4 SECTIONS / 16 TOPICS

14.1 Pattern Matching

- Naive String Matching
- Knuth-Morris-Pratt (KMP)
- Z-Function
- Rabin-Karp / Polynomial String Hashing
- Double Hashing (collision safety)

14.2 Palindrome Techniques

- Manacher's Algorithm
- Palindromic Substrings Counting

14.3 Advanced String Structures

- Trie of Strings
- Suffix Array (construction, LCP array)
- Suffix Automaton
- Suffix Tree (conceptual)
- Aho-Corasick Automaton (multi-pattern matching)

14.4 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Find Pattern in Text?	**KMP / Z-Function**
Compare Substrings Fast?	**String Hashing / Suffix Array**
Multiple Patterns at Once?	**Aho-Corasick**
Longest Palindromic Substring?	**Manacher's**

15 Computational Geometry

4 SECTIONS / 14 TOPICS

15.1 Basics

- Points, Vectors, Cross/Dot Product
- Orientation (CW/CCW/Collinear)
- Distance Formulas
- Line & Segment Representation

© 2026 Mindflare · IT Department Society

15.2 Polygon Algorithms

- Convex Hull (Graham Scan)
- Convex Hull (Andrew's Monotone Chain)
- Point-in-Polygon Test
- Polygon Area (Shoelace Formula)

15.3 Intersection & Proximity

- Line/Segment Intersection
- Closest Pair of Points (Divide & Conquer)
- Circle-Line/Circle-Circle Intersection

15.4 Problem-Solving Patterns

PATTERN CHEAT SHEET

When you see...	Reach for...
Smallest Enclosing Shape?	**Convex Hull**
Do Segments Cross?	**Orientation Tests**
Closest Pair Among n Points?	**Divide & Conquer**

16 ICPC Team & Contest Skills

3 SECTIONS / 12 TOPICS

16.1 Team Strategy

- Role Division (coder / reader-solver / debugger)
- Problem Triage (reading all problems first, ranking by difficulty)
- Whiteboard Communication
- Computer-Time Sharing

16.2 Execution Skills

- Fast, Correct Implementation Under Time Pressure
- Template/Snippet Libraries (pre-written DSU, segment tree, etc.)
- Edge Case Checklists
- Stress Testing Mid-Contest

16.3 Practice Discipline

- Virtual Contests (Codeforces Gyms, ICPC Regional Archives)
- Upsolving Every Contest
- Editorial + Alternate Solution Review
- Rating/Difficulty-Tagged Practice Sets

© 2026 Mindflare · IT Department Society

Suggested Progression

A study order that front-loads the highest-return material and keeps the slow-maturing skills (DP, math, team play) running alongside.

Step	Modules	Focus
1	01-06	Foundation phase. Build fluency here before touching graphs and DP seriously.
2	07-09	Trees and graph traversal / shortest paths. The highest ROI section for early contests.
3	11 (DP)	Start early alongside 07-09, since DP thinking takes the longest to mature. Revisit repeatedly.
4	10, 12	Advanced graphs and data structures, once you are comfortable with 08-09.
5	13 (Math)	Weave in continuously from day one. Modular arithmetic and combinatorics show up everywhere.
6	14-15	String algorithms and geometry. Typically lower frequency in ICPC regionals, so do them after core graph, DP and data structures are solid.
7	16	Start practicing team skills from your very first mock contest, not at the end.

© 2026 Mindflare · IT Department Society